

Lab 02 - Image and Video Data Representation

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 02: AI Image Generation, Editing and Visual Enhancement Outcome: ELO2 | In-class time: 15 minutes of scheduled practical time

Competency	Statement
Ability A2	Apply the principles of processing, filtering and analysis methods for video data
Knowledge K3	Methods to represent image and video data

Scenario

Before you can filter, edit or generate anything you have to know exactly what you are holding. Interrogate a still and a clip as data: array shape, dtype, channel order, colour space, frame rate and uncompressed bit-rate - the numbers that decide every later architecture choice. Classroom core (15 minutes): run the supplied measurement script, inspect the still/clip report and explain channel order and byte rate. The full implementation walkthrough and extra media trials are extension work.

What you will produce

A measurement report (out/representation-report.json) recording shape, dtype, channel means, HSV ranges, frame count, FPS, duration and the computed uncompressed byte rate for the supplied clip.

Tools: Python 3 | opencv-python | NumPy | offline after dependencies are installed; no paid service

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3, opencv-python, numpy.

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are **not** in that wheel and are deliberately avoided: cv2.TrackerCSRT_create / cv2.TrackerKCF_create and the cv2.quality SSIM module, both of which live in opencv-contrib-python. Where SSIM is needed the course ships its own NumPy implementation.

Workflow

1. Load the still and print shape and dtype
2. Split BGR, then convert to Gray and HSV
3. Measure per-channel statistics
4. Open the clip and read FPS and frame count

5. Compute the uncompressed byte rate

Files in this folder

Path	What it is
reference/nq-product-flatlay.png	Synthetic product flat-lay reference still, 1280x720 (deterministically rendered with OpenCV - SIMULATED classroom asset, not a photograph)
reference/nq-shelf-pan.mp4	Synthetic 6-second 640x360 25 fps shelf-pan clip (deterministically rendered animation - SIMULATED, not generative-model output)
data/expected-representation.json	Reference answers for the shape, dtype, FPS and frame-count checks so you can self-verify without the trainer
represent.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

Steps

1. Before class, install dependencies online or from a trainer-supplied wheelhouse. At class time use the prepared environment offline. On Windows activate with .venv\Scripts\Activate.ps1; the shell commands below are for macOS/Linux. Create and activate the lab environment, then confirm the OpenCV version. Everything in this lab runs on the base opencv-python wheel - no contrib modules and no downloaded weights.

```
python3 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2; print(cv2.__version__)"
```

2. For the following Python fragments, open a notebook or persistent Python REPL, run `import cv2; img=cv2.imread("reference/nq-product-flatlay.png"); assert img is not None`. Shell commands labelled `python3 -c` are separate demonstrations and do not retain variables. Load the still with `cv2.imread` and print its shape and dtype. Record the three numbers: height, width and channel count.

```
python3 -c "import cv2;i=cv2.imread('reference/nq-product-flatlay.png');print(i.shape, i.dtype)"
```

3. Confirm the channel order is B, G, R - not R, G, B. Read the pixel at (row 10, col 10) in the top-left calibration patch, which is drawn pure blue. A BGR read returns a high first element; an RGB read would return a high third element.

```
python3 -c "import cv2;i=cv2.imread('reference/nq-product-flatlay.png');print('pixel (10, 10) =', i[10,10])"
```

4. Split the image into its three channels with `cv2.split` and print the mean of each. Note which channel has the largest mean intensity - this is what a naive RGB threshold would be keying on.

```
b, g, r = cv2.split(img)
print('B mean %.1f G mean %.1f R mean %.1f' % (b.mean(), g.mean(), r.mean()))
```

5. Convert to grayscale with `cv2.COLOR_BGR2GRAY` and confirm the result is 2-D - the channel axis disappears. Grayscale is a weighted sum, not an average: OpenCV uses $0.299R + 0.587G + 0.114B$.

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
print(gray.shape, gray.dtype)
```

6. Verify that weighting yourself on the calibration patches. Compute the luma of a pure green patch by hand ($0.587 * 255 = 149.7$) and compare it against the grayscale value OpenCV produced. Locate a pure green pixel programmatically and report its coordinates; fail if the required patch is absent.

```
import numpy as np
ys,xs=np.where(np.all(img == (0,255,0),axis=2)); assert len(xs), 'missing green patch'
y,x=int(ys[0]),int(xs[0]); print(y,x,int(gray[y,x]))
```

7. Convert to HSV with `cv2.COLOR_BGR2HSV`. Print the min and max of each channel. Confirm the OpenCV 8-bit convention: H is 0-179 (degrees halved so it fits a byte), S and V are 0-255. Confusing this encoding with degree values causes incorrect thresholds.

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
for n, c in zip('HSV', cv2.split(hsv)):
    print(n, int(c.min()), int(c.max()))
```

8. Read the hue of the red, green and blue calibration patches. Red sits near $H=0$ (and wraps to 179), green near $H=60$, blue near $H=120$. Write the three measured values down; you will threshold on them in Lab 06. Find an exact pure-colour pixel for each patch and report coordinates; do not guess coordinates.

```
for name,bgr in [('red',(0,0,255)),('green',(0,255,0)),('blue',(255,0,0))]:
    ys,xs=np.where(np.all(img==bgr,axis=2)); assert len(xs), name+' patch missing'
    y,x=int(ys[0]),int(xs[0]); print(name,y,x,hsv[y,x].tolist())
```

9. Open the clip with `cv2.VideoCapture` and read its properties: frame width, frame height, FPS and frame count. Do not trust a filename or a container label for these.

```
cap = cv2.VideoCapture('reference/nq-shelf-pan.mp4')
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
print(w, h, fps, n, 'duration %.2fs' % (n / fps))
```

10. Count the frames you can actually read in a loop and compare that count with `CAP_PROP_FRAME_COUNT`. The property is backend-dependent and may be derived from metadata and can disagree with the decodable frame count - record the loop result and investigate any mismatch in work that matters.

```
read = 0
while True:
    ok, frame = cap.read()
    if not ok:
        break
    read += 1
print('reported', n, 'decoded', read)
```

11. Compute the uncompressed byte rate: width x height x channels x bytes-per-channel x fps (uint8 BGR uses 3 bytes/pixel). For the supplied 640x360 25 fps clip that is $640*360*3*25 = 17,280,000$ B/s = 17.3 MB/s. Then compute

the same figure for 1920x1080 at 30 fps and write both numbers down - Topic 5 uses them to justify sending events instead of frames.

```
for (W, H, F) in [(640, 360, 25), (1920, 1080, 30)]:
    print(W, H, F, '->', W * H * 3 * F / 1e6, 'MB/s uncompressed')
```

12. Measure the actual file size on disk and divide by the duration to get the encoded byte-rate (multiply by 8 for bits/s). Compare it with the uncompressed rate you just computed and state the compression ratio.

```
python3 -c "import os;print(os.path.getsize('reference/nq-shelf-pan.mp4'), 'bytes')"
```

13. Run the full report script. It performs every measurement above and writes out/representation-report.json.

```
python3 represent.py --image reference/nq-product-flatlay.png --video reference/nq-shelf-pan.mp4 --out out/representation-report.json
```

14. Compare your report against data/expected-representation.json. The shape, dtype, FPS, decoded frame count must match the fixed reference; compare rates/FPS with tolerance 0.01 and means with tolerance 0.1; the channel means may differ in the last decimal place across OpenCV builds.

```
python3 -c "import json;a=json.load(open('out/representation-report.json'));b=json.load(open('data/expected-representation.json'));keys=['image_shape','image_dtype','video_frames_decoded'];bad=[k for k in keys if k not in b or a.get(k)!=b[k]];bad += [k for k in ['video_fps','uncompressed_bytes_per_second'] if k not in b or abs(a[k]-b[k])>0.01];print(bad or 'ALL MATCH')"
```

15. Save out/representation-report.json and complete the evidence checklist.

Verify

out/representation-report.json exists and its image_shape, image_dtype, video_fps, video_frames_decoded and uncompressed_bytes_per_second fields match data/expected-representation.json, and you can state the OpenCV hue range from memory.

Expected outputs

- image_shape = [720, 1280, 3] and image_dtype = "uint8".
- Pixel (10,10) reads as a high-B, low-G, low-R triple, confirming BGR order.
- Measured HSV extrema lie within allowed uint8 ranges H 0-179, S/V 0-255; actual extrema need not reach those limits.
- video_fps = 25.0, 150 frames decoded, duration 6.00 s.
- uncompressed_bytes_per_second = 17280000 for the supplied clip.

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- ☐ out/representation-report.json attached or pasted.
- ☐ The measured hue values of the red, green and blue calibration patches.
- ☐ The two uncompressed byte-rate figures (640x360@25 and 1920x1080@30).
- ☐ The compression ratio of the supplied clip (encoded size vs uncompressed).
- ☐ A one-line statement of why CAP_PROP_FRAME_COUNT is not authoritative.

Troubleshooting

Symptom	What to do
cv2.imread returns None	The path is wrong or the file is unreadable - imread does not raise. Always guard with if img is None: raise SystemExit('cannot read <path>') and run from the lab folder, not from its parent.
Colours look wrong when displayed with matplotlib	matplotlib expects RGB; OpenCV gives BGR. Show with plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)).
Hue values above 179 or a hue of 255 for red	Check for HSV_FULL, wrong-channel interpretation, or a float image. Swapping RGB/BGR changes hue identity but does not exceed the uint8 H bound. Convert from the raw cv2.imread result and keep dtype uint8.
cap.get(cv2.CAP_PROP_FPS) returns 0	The backend could not read the container metadata. Inspect trusted timestamps/container metadata or re-copy the known reference asset. Never infer playback FPS by timing decode throughput. Do not time N frames as a substitute. A damaged MP4 may lack its moov atom.
Decoded frame count is one less than reported	For this supplied fixed asset, 150 decodable frames are required. Report both numbers and re-copy/recheck the input or decoder if they differ.

References used in this lab

- [S23] OpenCV 4.13 Python package, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92)*.

API-currency verification for every command taught: cv2.data.harcascades ships the frontal-face and eye cascades so no weights download is required; cv2.HOGDescriptor_getDefaultPeopleDetector() is built in; SIFT_create, ORB_create, FastFeatureDetector_create, cornerHarris, goodFeaturesToTrack, matchTemplate, createBackgroundSubtractorMOG2/KNN, meanShift, CamShift, calcOpticalFlowFarneback and PSNR are all present. IMPORTANT CORRECTION carried into the labs: cv2.TrackerCSRT_create and cv2.TrackerKCF_create are NOT in the base opencv-python wheel (they require opencv-contrib-python), and the cv2.quality SSIM module is also contrib-only - so the labs use cv2.TrackerMIL_create and a NumPy SSIM implementation to stay dependency-light and runnable offline.

- [S14] A. Mewada, M. A. Ansari, S. Ahmad and N. Singh (eds), 'AI-Generated Image and Video Synthesis', IEEE Press / Wiley, 2026, ISBN 9781394403110 - *Ch.1 pp.1-11*.

Historical evolution of synthetic media; role of data in training generative models; overview of major architectures; text-to-image, style transfer and super-resolution as distinct image-synthesis tasks; frame-based versus temporal video generation, motion transfer, and the temporal-consistency and realism challenges.

- [S18] Mewada et al. (2026), as above - *Ch.7 pp.98-100*.

Peak memory = parameter memory + activations (+ optimiser state during training); diffusion sampling latency proportional to S denoising steps for images and to S x F for video; moving weights from 32-bit to b-bit scales parameter memory by b/32; low-rank adapters train r(d+m) parameters versus d*m; energy per sample approximated by overhead factor x energy-per-FLOP x FLOPs-per-sample, with hardware, kWh, PUE, grid carbon intensity and joules-per-sample all reportable; Table 7.4 deployment decision matrix (low-latency image -> distilled sampler + INT8; edge device -> INT8 + PEFT adapters; high resolution -> latent diffusion + few-step sampler; short video -> temporal attention + distilled steps; brand/style control -> retrieval conditioning + filters; tight privacy -> local open model + provenance tracking).

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v11.0.